Esercizio gRPC: server Python, client Java

Advanced Computer Programming — tecnologie middleware

Tempo stimato: 45–60 minuti
23 ottobre 2026

1. Obiettivo

Realizzare un servizio gRPC **poliglotta**: il server è scritto in Python, il client in Java. I due processi non condividono alcuna riga di codice: l'unico elemento comune è il file `.proto`, dal quale ciascun linguaggio genera autonomamente i propri stub.

Lo scopo didattico non è la logica applicativa (banale di proposito), ma verificare di saper gestire le tre fasi tipiche di un'architettura middleware a chiamata remota: definizione del contratto in IDL, generazione del codice, implementazione di skeleton e stub.

2. Prerequisiti

Componente	Comando / artefatto
Python ≥ 3.9	`pip install grpcio grpcio-tools`
JDK ≥ 17	`java -version`
Maven ≥ 3.8	`mvn -version`

Il progetto Java richiede nel `pom.xml` le dipendenze `grpc-netty-shaded`, `grpc-protobuf`, `grpc-stub`, l'annotazione `javax.annotation-api` (per `@Generated` sulle classi prodotte) e i plugin `os-maven-plugin` e `protobuf-maven-plugin`. Verifica su *mvnrepository* le versioni correnti di `io.grpc` e di `protoc`: devono essere mutuamente compatibili.

3. Il contratto

Crea il file `proto/meteo.proto`, **identico** nei due progetti:

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "it.unina.acp.meteo";
option java_outer_classname = "MeteoProto";

package meteo;

service StazioneMeteo {
  // RPC unaria
  rpc RilevaTemperatura (CittaRequest) returns (RilevazioneReply);

  // RPC server-streaming
  rpc Monitora (CittaRequest) returns (stream RilevazioneReply);
}

message CittaRequest {
  string citta = 1;
  int32 campioni = 2;   // usato solo da Monitora
}

message RilevazioneReply {
  string citta = 1;
  double temperatura_c = 2;
  int64 timestamp_unix = 3;
}
```

Nota il doppio sistema di nomi: `temperatura_c` in `snake_case` nell'IDL diventa `getTemperaturaC()` in Java e resta `temperatura_c` in Python. È il generatore a imporre le convenzioni idiomatiche di ciascun linguaggio.

4. Generazione degli stub

Python — dalla radice del progetto Python:

```
python -m grpc_tools.protoc -I proto \
    --python_out=. --grpc_python_out=. \
    proto/meteo.proto
```

Produce `meteo_pb2.py` (messaggi) e `meteo_pb2_grpc.py` (servizio).

Java — colloca il `.proto` in `src/main/proto/` e lascia che sia `protobuf-maven-plugin` a invocare `protoc` durante `mvn compile`. Le classi finiscono in `target/generated-sources/protobuf/`.

5. Task 1 — Server Python

Implementa `server.py`. La classe di servizio deve ereditare dallo skeleton generato e sovrascrivere i metodi dichiarati nell'IDL.

```
import time, random
from concurrent import futures
import grpc
import meteo_pb2, meteo_pb2_grpc

class StazioneMeteoServicer(meteo_pb2_grpc.StazioneMeteoServicer):

    def RilevaTemperatura(self, request, context):
        # TODO: restituisci un RilevazioneReply con una temperatura
        # pseudo-casuale fra -5 e 40 gradi e il timestamp corrente
        ...

    def Monitora(self, request, context):
        # TODO: emetti request.campioni rilevazioni, una ogni secondo.
        # Suggerimento: questo metodo non fa return, ma ...
        ...

def serve():
    # TODO: crea il server con un ThreadPoolExecutor, registra il
    # servicer, metti in ascolto sulla porta 50051, avvia e attendi
    ...

if __name__ == "__main__":
    serve()
```

Vincolo: se `request.citta` è una stringa vuota, il server deve chiudere la chiamata con stato `INVALID_ARGUMENT` e un messaggio esplicativo, non sollevare un'eccezione Python generica.

6. Task 2 — Client Java

Implementa `MeteoClient.java` usando lo stub **bloccante**.

```java
package it.unina.acp.meteo;

import io.grpc.*;

public class MeteoClient {
    public static void main(String[] args) {
        // TODO: costruisci un ManagedChannel verso localhost:50051.
        // Il canale va dichiarato in chiaro (senza TLS): quale metodo
        // del builder serve?
        ManagedChannel channel = ...;

        try {
            StazioneMeteoGrpc.StazioneMeteoBlockingStub stub =
                StazioneMeteoGrpc.newBlockingStub(channel);

            CittaRequest req = CittaRequest.newBuilder()
                    .setCitta("Napoli")
                    .setCampioni(3)
                    .build();

            // TODO 2a: chiamata unaria, stampa il risultato

            // TODO 2b: chiamata a Monitora. Lo stub bloccante
            // restituisce un Iterator<RilevazioneReply>: iteralo e
            // stampa ogni rilevazione appena arriva

        } catch (StatusRuntimeException e) {
            // TODO: distingui almeno UNAVAILABLE da INVALID_ARGUMENT
        } finally {
            // TODO: chiudi il canale ordinatamente
        }
    }
}
```

7. Task 3 — Verifica della semantica di errore

Invia una `CittaRequest` con `citta` vuota e osserva che cosa riceve il client. Annota in due righe: quale campo dello `Status` trasporta il messaggio scritto dal server Python, e perché il client Java riesce a leggerlo pur non condividendo il codice del server.

8. Output atteso

```
[unaria]  Napoli: 21.4 °C  (t=1792345600)
[stream]  Napoli: 21.9 °C  (t=1792345601)
[stream]  Napoli: 20.7 °C  (t=1792345602)
[stream]  Napoli: 22.1 °C  (t=1792345603)
```

Le tre righe dello stream devono comparire **a distanza di un secondo l'una dall'altra**, non tutte insieme alla fine. Se arrivano in blocco, stai bufferizzando da qualche parte: cerca dove.

9. Domande di autovalutazione

1. Che cosa cambia, lato rete, fra lo stub bloccante e quello asincrono? Il numero di connessioni TCP aperte è lo stesso?
2. Il server Python usa un `ThreadPoolExecutor`: che succede se arrivano più chiamate `Monitora` concorrenti che thread disponibili?
3. Perché proto3 non distingue, per un campo scalare, fra "assente" e "valore di default"? Quale conseguenza ha sul controllo della città vuota del Task 1?
4. Se aggiungessi al messaggio `RilevazioneReply` un campo `string nota = 4;` e rigenerassi solo il client Java, il sistema continuerebbe a funzionare? E se eliminassi il campo 2?

10. Trappole frequenti

- Il nome della classe servicer in Python deve corrispondere a quello generato (`<Servizio>Servicer`): un errore di battitura non produce errori a compile-time, ma `UNIMPLEMENTED` a runtime.
- In Java, se le classi generate non compaiono, controlla che il `.proto` sia esattamente sotto `src/main/proto` e che `os-maven-plugin` sia dichiarato fra le `<extensions>`, non fra i `<plugins>`.
- `java_package` nell'IDL deve coincidere con il package effettivo dei sorgenti Java, altrimenti gli import non risolvono.
- Un `ManagedChannel` non chiuso lascia il thread di Netty attivo e il processo Java non termina.